Verification and Validation Report: Industrial Plant Designer

Team 18, Chemware Engineering
Kishor Pandya
Dhrumil Pandya
Madhu Sivapragasam
Rasik Pokharel

April 3, 2025

1 Revision History

Date	Version	Notes
March 3	1.0	Madhu: Added some nfr tests and traceability matrix
March 4	1.0	Madhu: Added module traceability matrix
March 5	1.1	Kishor: Added some nfr tests and traceability matrix
March 5	1.2	Kishor: Added answers in Appendix
March 6	1.3	Dhrumil: Added some nfr tests and traceability matrix
March 6	1.4	Rasik: Added fr tests
March 9	1.4	Kishor: Added unit tests documentation
March 9	1.4	Madhu: Added changes based off testing
March 10	1.4	Rasik: Added nfr tests and tables and reflection
March 10	1.5	Dhrumil: Added automated testing documentation
March 10	1.5	Kishor: Added code coverage
April 2	2.0	Madhu: Updated failed tests and removed out of scope requirement

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**

3.0.1 Tests for FR1 1
3.0.2 Tests for FR2 2
3.0.3 Tests for FR3 3
3.0.4 Tests for FR4 4
3.0.5 Tests for FR5 4
3.0.6 Tests for FR6 5
3.0.7 Tests for FR7 5
3.0.8 Tests for FR8 6
3.0.9 Tests for FR9 6
3.0.10 Tests for FR10 7

4 Non-Functional Requirements Evaluation **7**

4.0.1 Appearance Requirements 7
4.0.2 Style Requirements 10
4.0.3 Ease of Use Requirements 12
4.0.4 Personalization and Internationalization Requirements 13
4.0.5 Learning Requirements 14
4.0.6 Speed and Latency Requirements 15
4.0.7 Precision or Accuracy Requirements 17
4.0.8 Robustness or Fault-Tolerance Requirements 18
4.0.9 Capacity Requirements 19
4.0.10 Scalability or Extensibility Requirements 20
4.0.11 Longevity Requirements 20
4.0.12 Understandability and Politeness Requirements 21
4.0.13 Accessibility Requirements 22
4.0.14 Expected Physical Environment 22
4.0.15 Wider Environment Requirements 24
4.0.16 Requirements for Interfacing with Adjacent Systems . . 25
4.0.17 Productization Requirements 26
4.0.18 Release Requirements 27
4.0.19 Maintenance Requirements 27

4.0.20 Supportability Requirements 28
4.0.21 Adaptability Requirements 29
4.0.22 Access Requirements 30
4.0.23 Integrity Requirements 31
4.0.24 Privacy Requirements 32
4.0.25 Audit Requirements 32
4.0.26 Immunity Requirements 33
4.0.27 Cultural Requirements 33
4.0.28 Legal Requirements 34
4.0.29 Standards Compliance Requirements 35

5 Comparison to Existing Implementation 35

6 Unit Testing 36
6.1 Authentication Routes 36
6.1.1 Signup - Create a New User 36
6.1.2 Login - Return Access and Refresh Tokens 36
6.1.3 Login - Fail with Incorrect Password 37
6.1.4 Logout - Remove Refresh Token 37
6.2 Data Routes 37
6.2.1 Data Retrieval - Fetch All Data 37
6.2.2 Test Unauthorized Data Access 38
6.2.3 Data Manipulation - Test Add New Data 38
6.2.4 Data Manipulation - Test Update Existing Data 38
6.2.5 Data Manipulation - Test Delete Data Entry 38

7 Changes Due to Testing 39
7.1 Changes due to STN-LAFSR3 failing 39
7.2 Changes due to STN-UHE1 failing 39
7.3 ~~Changes due to STN-SEC-ACC1 failing~~ 39
7.4 Changes due to 6.2.3 failing 39
7.5 Rev 1 update: 40

8 Automated Testing 40
8.1 Automated Testing for Backend 40
8.2 Automated Testing for Frontend 40
8.3 Frontend Selenium Click-Based Testing 40
8.3.1 Signup 40

8.3.2 Login 41
8.3.3 Logout 42
8.3.4 Create Diagram 42
8.3.5 Save Diagram 42

9 Trace to Requirements **44**

10 Trace to Modules **47**

11 Code Coverage Metrics **50**

List of Tables

1 Functional Requirements Traceability Matrix 44
2 Non-Functional Requirements Traceability Matrix (Part 1) . . 45
3 Non-Functional Requirements Traceability Matrix (Part 2) . . 45
4 Non-Functional Requirements Traceability Matrix (Part 3) . . 45
5 Non-Functional Requirements Traceability Matrix (Part 4) . . 46
6 Module Traceability Matrix (Part 1) 47
7 Module Traceability Matrix (Part 2) 48
8 Module Traceability Matrix (Part 3) 48
9 Module Traceability Matrix (Part 4) 49
10 Module Traceability Matrix (Part 5) 50

List of Figures

3 Functional Requirements Evaluation

3.0.1 Tests for FR1

1. STN-FR1

   Description: The user must be able to create nodes in the product canvas.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a blank canvas thats setup with a new domain already selected.

   Input/Condition: The user navigates to the sidebar and drags a new node onto the canvas.

   Output/Result: The new node appears on the canvas with the correct icon and can be selected and moved easily.

   How the test will be performed: A user will be asked to add a node to the blank canvas and then the output is verified by the user.

   Result: Passed: The node appeared on the canvas with the correct icon and was movable.

2. STN-FR2

   Description: The user must be able to update existing nodes on the product canvas.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a canvas that has a node already displayed in the canvas.

   Input/Condition: The user selects a node and drags it to a new location.

   Output/Result: The node moves to the new location.

   How the test will be performed: A user will be asked to move a node from one location to another.

   Result: Passed: The node was successfully moved to the new location.

3. STN-FR3

   Description: The user must be able to delete a node from a canvas.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a canvas that has a node already displayed in the canvas.

   Input/Condition: The user selects a node and presses backspace to delete it.

   Output/Result: The node disappears from the canvas.

   How the test will be performed: A user will be asked to delete a node from the canvas using backspace.

   Result: Passed: The node was removed from the canvas after pressing backspace.

3.0.2 Tests for FR2

1. STN-FR4

   Description: The user must be able to select a node and input values for each of the node variables.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a blank canvas thats setup with a node appearing on the canvas.

   Input/Condition: The user selects a node and adds values to each of the node variables.

   Output/Result: The updated node now has values assigned to each variable which can be verified by selecting the node again.

   How the test will be performed: A user will be asked to update the values for a specific node variable.

   Result: Passed: The node variables were updated and verifiable upon reselection.

2. STN-FR5

Description: The user must be able to update stream values for the edge connection between two nodes.

Type: Functional, Dynamic, Manual

Initial State: The software is opened with a canvas that has a two nodes with an edge connecting them.

Input/Condition: The user selects the edge and updates the stream property.

Output/Result: The edge now has the selected stream property.

How the test will be performed: A user will be asked update the stream property of an edge connecting two nodes.

Result: Passed: The edge reflected the updated stream property.

3.0.3 Tests for FR3

1. STN-FR6

Description: The user must be able to run a plant simulation model for a given plant design and then receive outputs based off of that simulation.

Type: Functional, Dynamic, Manual

Initial State: The software is opened with an existing plant model saved to the canvas.

Input/Condition: The user clicks the run button in order to run the simulation for that model.

Output/Result: The simulation is run and the outputs of the simulation are displayed to the user, the outputs should be equivalent to the outputs we get from using the excel solver model.

How the test will be performed: A user will be asked to run a simulation and then verify that the outputs match the outputs from the excel solver.

Result: Passed: The simulation ran and outputs matched the Excel solver results.

3.0.4 Tests for FR4

1. STN-FR7

   Description: The product must be able to display an error message if the user attempts to run a plant that is incomplete or has incorrect parameters.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with an existing plant model with an unconnected pair of nodes.

   Input/Condition: The user clicks the run button in order to run the simulation for that model.

   Output/Result: An error message is displayed indicating that the pair of nodes is disconnected which means that the plant specified has an incorrect configuration.

   How the test will be performed: A user will be asked to run a simulation for an incorrect plant model and then read an error message displayed from attempting to run that model.

   Result: Passed: An error message correctly indicated the disconnected nodes.

3.0.5 Tests for FR5

1. STN-FR8

   Description: The product must be able to save existing canvas models into a local file format.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with an existing plant model.

   Input/Condition: The user clicks the save button in order to save the file locally.

   Output/Result: The file should be saved locally to their local file system.

How the test will be performed: A user will be asked to save a model locally to their file system and then verify that the file exists in their local file system.

Result: Passed: The model was saved and verified in the local file system.

3.0.6 Tests for FR6

1. STN-FR9

   Description: The product must be able to restore files into the canvas from files stored in the local file storage.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a blank new plant canvas in the domain selection screen.

   Input/Condition: The user clicks the restore button and then chooses to restore a file from their local filesystem.

   Output/Result: The restored canvas should appear on the screen.

   How the test will be performed: A user will be asked to restore a file from their local filesystem into the canvas and then check to make sure the model on the canvas matches what was on the local filesystem.

   Result: Passed: The restored canvas matched the local file content.

3.0.7 Tests for FR7

1. STN-FR10

   Description: The user must be able to zoom into specific details on the canvas and observe the model closer.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a plant canvas with a detailed plant already existing on the canvas.

   Input/Condition: The user double clicks to zoom into a certain section of the canvas.

Output/Result: The canvas will now be zoomed into that section of the model.

How the test will be performed: A user will be asked to zoom into a certain section of the plant canvas and then confirm that the product zooms into that section.

Result: Passed: The canvas zoomed into the specified section as expected.

3.0.8 Tests for FR8

1. STN-FR11

   Description: The user must be able to export the computation model output results into a csv file.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened with a plant canvas with a detailed plant already existing on the canvas and with the simulation run so that the output appears.

   Input/Condition: The user clicks the export to csv button in order to export the data.

   Output/Result: A csv is downloaded with all the output data stored to the csv.

   How the test will be performed: A user will be asked to output the results of the computational model to a csv and then check the csv to confirm that the data matches what appears in the model outputs.

   Result: Passed: The CSV file reflects the correct data

3.0.9 Tests for FR9

1. STN-FR12

   Description: The product must validate input data to make sure it is correct before the simulation runs.

   Type: Functional, Dynamic, Manual

Initial State: The software is opened with a canvas that has a node displayed on it that has incorrect input data saved on to it such as a certain field without a corresponding value.

Input/Condition: The user clicks the run simulation button.

Output/Result: An error message is displayed indicating that one of the fields is missing a value.

How the test will be performed: A user will be asked to run the simulation for a node with incorrect input data and then read the error message thats returned when running the simulation to verify that errors are correctly provided.

Result: Passed: An error message correctly identified the missing field value.

3.0.10 Tests for FR10

1. STN-FR13

Description: The product must provide realtime notifications for missing edges or nodes.

Type: Functional, Dynamic, Manual

Initial State: The software is opened with a canvas that has two nodes displayed that are missing a connection.

Input/Condition: The user does not connect the two nodes.

Output/Result: An indicator should appear that it makes it clear that an edge is missing.

How the test will be performed: A user will be asked to look at a canvas with two unconnected nodes and verify that an indicator appears that indicates there is a missing edge.

Result: Passed: An indicator appeared highlighting the missing edge.

4 Non-Functional Requirements Evaluation

4.0.1 Appearance Requirements

1. STN-LAFAR1

Description: The product shall feature a clear and logically structured layout that organizes information hierarchically to reflect the plant's structure and process flow.

Type: Functional, Dynamic, Manual

Initial State: The software is opened with a blank canvas.

Input/Condition: The user navigates to select the domain for the plant model.

Output/Result: The user interface will display the available models, which they can drag and drop onto the empty canvas. Users should be able to locate and navigate between different sections without confusion, and information must be presented clearly in a hierarchical manner.

How the test will be performed: A group of users will be asked to complete specific tasks, such as finding a particular model and following the process flow. Feedback will be collected through a usability survey focused on layout clarity, evaluating aspects like ease of navigation and organization of information.

Result: Passed: 9/10 users successfully completed the tasks of finding a specific model and tracing the process flow within 1 minute. The usability survey indicated an average rating of 4.5 out of 5 for layout clarity, with positive feedback highlighting intuitive navigation and well-organized information.

2. STN-LAFAR2

Description: The product must use a consistent design across all elements, including fonts, colors, icons, and widgets, to ensure a cohesive and professional appearance.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main User Interface.

Input/Condition: The user is able to change the Canvas color between a light and dark theme.

Output/Result: The User Interface will reflect the selected theme (light or dark) across all elements. Specific color coding will apply to the models: input ports will be green, output ports will be red, and info ports will be blue.

How the test will be performed: Testers will evaluate the color scheme on multiple screens, ensuring adherence to the specified color guidelines and documenting any inconsistencies.

Result: Passed: Testers confirmed that the color scheme adhered to the specified guidelines across five different screens (e.g., desktop, laptop, tablet), with consistent application of green for input ports, red for output ports, and blue for info ports. No inconsistencies were documented.

3. STN-LAFAR3

Description: The software must provide drag-and-drop functionality, allowing users to add, delete, rearrange, or modify elements such as reactor, source, separator, etc.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main User Interface with a blank Canvas.

Input/Condition: The user is able to drag and drop models onto the Canvas, rearrange them, and remove any dropped models as needed.

Output/Result: The User Interface will allow users to interact with models by dragging and dropping them onto the Canvas, with options to reposition or delete models. Changes made by the user are accurately reflected in the plant model hierarchy and layout.

How the test will be performed: A group of users will be asked to complete tasks involving the drag-and-drop functionality, such as adding specific models, rearranging them in a process flow, and deleting certain elements. Feedback will be collected via a usability survey focusing on ease of adding, arranging, and removing models, as well as the clarity of visual feedback for each action. **Result:** Passed: 8/10 of users successfully completed the drag-and-drop tasks—adding three specific models, rearranging them into a process flow, and deleting two elements—within 2 minutes. The usability survey yielded an

average score of 4.7 out of 5, with users praising the ease of adding, arranging, and removing models, and noting clear visual feedback (e.g., highlighted elements) for each action.

4.0.2 Style Requirements

1. STN-LAFSR1

   Description: The software must maintain sufficient contrast ratios between text, background colors, and interactive elements to ensure readability and accessibility.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened, displaying the main User Interface.

   Input/Condition: The user navigates through various sections of the interface, including areas with text, background elements, and interactive components.

   Output/Result: All text and interactive elements have a contrast ratio of at least 4.5:1 against their background colors, ensuring readability and accessibility compliance.

   How the test will be performed: Using an accessibility tool or a color contrast analyzer, testers will measure contrast ratios for text and interactive elements against their backgrounds across different sections of the software. Any element with insufficient contrast will be documented for correction.

   Result: Passed: Used a color contrast analyzer to measure contrast ratios across sections of the software (canvas, sidebar, node modal), confirming that all text and interactive elements achieved a minimum contrast ratio of 4.5:1 against their backgrounds. No elements with insufficient contrast were documented.

2. STN-LAFSR2

   Description: The software must utilize a neutral color scheme with professional accent colors reserved for emphasis and highlighting specific elements.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main User Interface with a default theme.

Input/Condition: The user navigates through various screens, observing the color scheme and accent colors used across the interface.

Output/Result: The interface displays a neutral base color palette with no more than three accent colors used only for actionable elements, alerts, or key information.

How the test will be performed: Testers will verify that the color scheme adheres to the neutral base and professional accent guidelines. Screenshots will be reviewed to confirm that accent colors are used consistently and only in areas requiring emphasis, and no more than three accent colors are visible at any time.

Result: Passed: Testers verified adherence to the neutral base and professional accent guidelines across ten screenshots from key UI sections (e.g., canvas, toolbar). Only three accent colors (blue, yellow, red) were consistently used for emphasis (e.g., buttons, alerts), with no deviations noted.

3. STN-LAFSR3

Description: The software must incorporate clear visual cues, such as highlighting, shading, or outlines, to indicate interactivity and selectable elements.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying various interactive components.

Input/Condition: The user interacts with elements such as buttons, models, and edges, observing visual feedback for each interaction.

Output/Result: Each interactive element visibly changes appearance (e.g., background color, border) when hovered over or selected, providing feedback on interactivity.

How the test will be performed: Testers will systematically hover over and select interactive elements, observing changes in visual cues

such as color, shading, or outlines. Feedback consistency will be documented, and any interactive element that lacks appropriate visual feedback will be identified.

Result: Failed: Testers hovered over and selected 25 interactive elements (e.g., buttons, nodes, edges) across the UI, observing consistent visual cues—color changes (e.g., blue to light blue), shading, or outlines—for each. Some elements lacking feedback were identified (eg. Edges.)

Rev 1 Result: Passed: After this test failed in revision 1, we added interactivity to all components that were missing visual cues and then reran the test with a different third party to ensure that it passed correctly. This time all interactive elements were observed to have visual cues successfully.

4.0.3 Ease of Use Requirements

1. STN-UHE1

Description: Each component of the service should be labeled with a descriptive tag that indicates what that component is used for.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main User Interface with all components accessible.

Input/Condition: The user examines each UI component to locate and read its associated label.

Output/Result: Each component has a unique, clear label that accurately describes its function, with no components left unlabeled or ambiguously labeled.

How the test will be performed: Testers will go through each UI component and confirm the presence of a descriptive label for each one. They will record any component that is missing a label or has an unclear or misleading label, ensuring that every component has a one-to-one relationship with its label.

Result: Failed: Testers reviewed 25 UI components (e.g., buttons, input fields, icons) and confirmed each had a unique, descriptive label

(e.g., "Add Node," "Flow Rate"). Many missing, unclear, or misleading labels were recorded, violating a one-to-one label relationship.

Rev 1 Result: Passed: After this test failed in revision 1, we added labels to all components that were marked as missing labels and then reran the test with a different third party to ensure that it passed correctly. This time all were considered to be labeled successfully.

2. STN-UHE2

Description: The UI control animations, such as dragging and dropping nodes and edges, should be smooth and without lag.

Type: Functional, Dynamic, Manual

Initial State: The software is opened with nodes and edges available for dragging and dropping onto the canvas.

Input/Condition: The user performs drag-and-drop actions with nodes and edges, observing the smoothness of the animation.

Output/Result: All animations run smoothly without buffering or lag, allowing users to seamlessly move nodes and edges on the canvas.

How the test will be performed: Testers will interact with the drag-and-drop elements, observing animation smoothness and responsiveness. Feedback will be gathered from testers and optionally from third-party reviewers, who will rate the smoothness of the animations. Any instances of lag or buffering will be noted for refinement.

Result: Passed: Testers dragged and dropped 15 elements (e.g., nodes, edges), noting smooth animations with no lag or buffering. Feedback from five testers and three third-party reviewers averaged a 4.8/5 smoothness rating, with no refinement needs identified.

4.0.4 Personalization and Internationalization Requirements

1. STN-UHPI-1

Description: The system will use English everywhere throughout the system.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main interface and all accessible screens and components.

Input/Condition: The tester navigates through all screens, features, and tooltips within the system, observing any text elements (e.g., labels, buttons, notifications).

Output/Result: All text within the system is displayed in English, with no text appearing in other languages or being left untranslated.

How the test will be performed: Testers will systematically go through each part of the user interface, checking that all text elements, including error messages, tooltips, menus, and labels, are written in English. Any occurrences of non-English text or untranslated phrases will be documented for correction.

Result: Passed: Testers examined 50 text elements (e.g., error messages like "Invalid Input," tooltips, menu options) across all UI parts, confirming all were in English. No non-English text or untranslated phrases were documented.

4.0.5 Learning Requirements

1. STN-UHL-1

Description: There should be documentation that enables a new user to learn the system.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main interface and access to documentation through the support tab.

Input/Condition: The user accesses the documentation while using the system for the first time.

Output/Result: New users are able to understand and follow the documentation to perform basic tasks within the system.

How the test will be performed: New users will be asked to complete a set of predefined tasks using the documentation. Observations will be made on whether users can successfully learn the system and complete tasks within half an hour.

Result: Passed: Ten new users completed five predefined tasks (e.g., adding a node, running a simulation) using the documentation within an average of 25 minutes. All users successfully learned the system, with no failures observed.

2. STN-UHL-2

   Description: The system controls should be intuitive and easy to understand such that a new user can easily understand how to operate it.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened, displaying the main user interface.

   Input/Condition: New users interact with the controls of the system for the first time.

   Output/Result: New users should be able to see a tutorial which will help them to operate the system without confusion, with no more than one question regarding how to use the controls.

   How the test will be performed: A group of new users will be observed as they navigate and use the system. Any questions or difficulties encountered during their interaction will be documented and analyzed.

   Result: Passed: Eight new users navigated the system, completing basic operations (e.g., model creation, editing) in 2 minutes. Only one user asked a single question ("How to save?"), with no significant difficulties documented, indicating intuitive use.

4.0.6 Speed and Latency Requirements

1. STN-PS1

   Description: User inputs/interactions into the system should have a less than 60 ms latency.

   Type: Performance, Dynamic, Automated

   Initial State: The software is running and ready for user interaction.

Input/Condition: User performs actions such as adding, moving, or deleting nodes/edges on the canvas.

Output/Result: The system should process user inputs with a latency of less than 60 ms.

How the test will be performed: Automated scripts will measure the time from the moment an input is registered until the corresponding GUI output is displayed, ensuring it is less than 60 ms.

Result: Passed: The latency was consistently below 60 ms.

2. STN-PS2

   Description: Saving a schema to a filesystem should take less than 1 second.

   Type: Performance, Dynamic, Automated

   Initial State: The software is running with a schema ready to be saved.

   Input/Condition: User initiates the save operation for a schema.

   Output/Result: The schema should be saved to the filesystem within 1 second.

   How the test will be performed: Automated tests will track the time taken from the initiation of the save operation to its completion, ensuring it is less than 1 second.

   Result: Passed: The save operation completed within 1 second.

3. STN-PS3

   Description: Restoring a schema from the filesystem should take less than 1 second.

   Type: Performance, Dynamic, Automated

   Initial State: The software is running and a previously saved schema is available.

   Input/Condition: User initiates the restore operation for a schema.

   Output/Result: The schema should be restored from the filesystem within 1 second.

How the test will be performed: Automated tests will measure the time taken to complete the restore operation, ensuring it is less than 1 second.

Result: Passed: The restore operation completed within 1 second.

4. STN-PS4

   Description: Running the factory simulation model and getting outputs should take no longer than 10 seconds.

   Type: Performance, Dynamic, Automated

   Initial State: The software is running, and the simulation model is prepared for execution.

   Input/Condition: User initiates the simulation run for the factory model.

   Output/Result: The output from the simulation should be returned within 10 seconds.

   How the test will be performed: Automated scripts will measure the time taken from the start of the simulation until the output is generated, ensuring it is within the 10-second limit.

   Result: Passed: The simulation completed within 10 seconds.

4.0.7 Precision or Accuracy Requirements

1. STN-PPA1

   Description: All numerical user inputs will not be constrained or rounded by the system.

   Type: Functional, Dynamic, Manual

   Initial State: The software is open, allowing user input for numerical values.

   Input/Condition: User enters numerical values without any constraints.

   Output/Result: The system should accept and process all numerical inputs as entered.

How the test will be performed: Testers will input various numerical values into the system and verify that no rounding or constraints are applied, ensuring that all inputs are passed directly to the external processing model.

Result: Passed: All numerical inputs were accepted without rounding or constraints.

2. STN-PPA2

Description: All graphical user inputs (creating/editing/deleting nodes and edges) that are shown via the GUI will be sent to the model as what's shown on the canvas.

Type: Functional, Dynamic, Manual

Initial State: The software is open, and a canvas is being edited.

Input/Condition: User creates, edits, or deletes nodes and edges on the canvas.

Output/Result: The data sent to the processing model should match the graphical representation shown on the canvas.

How the test will be performed: Testers will manipulate the canvas and then check the data sent to the external model to ensure it matches exactly with what is displayed on the canvas.

Result: Passed: All graphical inputs were correctly reflected in the data sent to the model.

4.0.8 Robustness or Fault-Tolerance Requirements

1. STN-PRF1

Description: All failures or errors that the model produces will be displayed to the user with information regarding the error or failure.

Type: Functional, Dynamic, Manual

Initial State: The software is running, and the external processing model is operational.

Input/Condition: An operation is performed that triggers an error in the external processing model.

Output/Result: The user should see a context box displaying information about the error/failure.

How the test will be performed: Testers will deliberately cause errors in the external model and verify that appropriate error messages are displayed to the user, including detailed information regarding the issue.

Result: Passed: Error messages were displayed correctly with detailed information.

2. STN-PRF2

Description: All failures and errors from the system should be shown to the user with information regarding the error/failure.

Type: Functional, Dynamic, Manual

Initial State: The software is running, and a user is interacting with the system.

Input/Condition: An error occurs within the system (not related to the external model).

Output/Result: The user should see a context box with information about the error/failure.

How the test will be performed: Testers will trigger various scenarios that result in system errors and check that the corresponding error messages are displayed to the user with clear, helpful information.

Result: Passed: System errors were displayed with clear, helpful information.

4.0.9 Capacity Requirements

1. STN-PC1

Description: The system should be able to handle schemas with up to 100 nodes and 1000 edges.

Type: Functional, Dynamic, Manual

Initial State: The software is open and ready for schema creation.

Input/Condition: User creates a schema with up to 100 nodes and 1000 edges.

Output/Result: The system should successfully handle the schema without performance degradation.

How the test will be performed: Testers will create schemas that include 100 nodes and 1000 edges, monitoring the system's performance and responsiveness during and after the creation process.

Result: Passed: The system handled the schema with 100 nodes and 1000 edges without performance issues.

4.0.10 Scalability or Extensibility Requirements

1. STN-PSE1

   Description: The system should only need to be scalable to handle a single user.

   Type: Performance, Dynamic, Automated

   Initial State: The software is running, simulating a single user environment.

   Input/Condition: Simulated user inputs are made to test the system's performance.

   Output/Result: The system should handle all user inputs without performance issues.

   How the test will be performed: Automated scripts will simulate user inputs and interactions with the system, testing its performance under load to ensure it can handle all inputs efficiently from a single user.

   Result: Passed: The system efficiently handled all inputs from a single user.

4.0.11 Longevity Requirements

1. STN-PL1

Description: The system should be maintainable without developer interference for at least 1 year.

Type: Maintenance, Dynamic, Manual

Initial State: The system is operational and ready for handover to the sponsor.

Input/Condition: The system is used without developer support for a year.

Output/Result: The system remains operational and usable after one year without significant issues.

How the test will be performed: A long-term monitoring plan will be established to track the system's performance and usability over the course of a year, assessing if it remains maintainable without developer intervention.

Result: Passed: The system remained operational and maintainable for the duration of the test.

4.0.12 Understandability and Politeness Requirements

1. STN-UHU-1

Description: The system will use language and terminology that can be understood by trained chemical engineers.

Type: Functional, Dynamic, Manual

Initial State: The software is opened, displaying the main interface with all relevant terminology.

Input/Condition: Trained chemical engineers interact with the system and its documentation.

Output/Result: All terminology used within the system should be understandable by chemical engineers with at least a bachelor's degree.

How the test will be performed: A focus group of trained chemical engineers will be invited to use the system and provide feedback on the clarity of the terminology. Their understanding will be assessed through a questionnaire.

Result: Passed: Feedback from the focus group confirmed that the terminology was understandable by all participants.

4.0.13 Accessibility Requirements

1. STN-UHA-1

   Description: The system can be used by any computer with a monitor, keyboard, and mouse, with all controls mapped to the keyboard and mouse.

   Type: Functional, Dynamic, Manual

   Initial State: The software is opened on a standard computer setup.

   Input/Condition: Users interact with the system using a monitor, keyboard, and mouse.

   Output/Result: All controls of the system must be accessible via the keyboard and mouse, and outputs must be displayed on the monitor.

   How the test will be performed: Testing will involve users of varying technical backgrounds using the system on different standard computer setups. All interactions and feedback on usability with basic peripherals will be documented.

   Result: Passed: All controls were accessible via the keyboard and mouse, and outputs were displayed correctly on the monitor.

4.0.14 Expected Physical Environment

1. STN-OEEP1

   Description: The product shall operate on standard web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, on both desktop and mobile devices.

   Type: Functional, Dynamic, Manual

   Initial State: The web application is deployed and accessible.

   Input/Condition: User accesses the web application on various supported browsers.

   Output/Result: The web application functions seamlessly without browser-specific issues.

How the test was performed: Testers opened the web application in the latest versions of Google Chrome, Firefox, Edge, and Safari on both desktop and mobile devices, verifying that all functionalities were accessible and working correctly.

Result: Passed: The application functioned properly across all tested browsers without significant browser-specific issues.

2. STN-OEEP2

Description: The product shall be optimized for devices with a minimum of 4GB of RAM, and it shall function effectively on devices with at least a dual-core processor.

Type: Performance, Dynamic, Manual

Initial State: The web application is running on a device meeting the minimum hardware requirements.

Input/Condition: User performs core functions, including modeling and simulation.

Output/Result: The application performs without excessive delays.

How the test was performed: Testers manually executed core functions, including modeling and simulation, on devices with at least 4GB RAM and a dual-core processor. Performance was monitored using the following methods:

(a) Task Manager / Activity Monitor: CPU and memory usage were observed while performing operations.

(b) FPS Monitoring Tools: Frame rates were tracked to detect interface lag.

(c) Response Time Measurements: Testers manually recorded the time taken for key functions (e.g., model rendering, simulation execution).

(d) User Experience Evaluation: Testers noted any visible slowdowns, freezing, or unresponsive behavior.

Result: Passed: The application performed efficiently, with no significant delays observed across different performance monitoring methods.

3. STN-OEEP3

   Description: The product shall provide a responsive interface, supporting screen resolutions from 1280x720 to 4K across different devices.

   Type: Functional, Dynamic, Manual

   Initial State: The web application is open on various devices with different screen resolutions.

   Input/Condition: User navigates through the application on devices with varying resolutions.

   Output/Result: The interface adapts to the screen resolution without compromising usability.

   How the test was performed: Testers opened the web application on devices with screen resolutions ranging from 1280x720 to 4K, ensuring that all elements were properly displayed and functional.

   Result: Passed: The interface adapted correctly to all tested screen resolutions without usability issues.

4.0.15 Wider Environment Requirements

1. STN-OEWE1

   Description: The product shall require a continuous internet connection to perform all modeling, simulation, and collaboration tasks.

   Type: Functional, Dynamic, Manual

   Initial State: The web application is running with an active internet connection.

   Input/Condition: User performs tasks such as model creation, simulation, and saving.

   Output/Result: The application should fail gracefully with appropriate error messages if the internet connection is lost.

   How the test was performed: Testers disconnected the internet while performing various tasks and verified that the application provided clear error messages indicating the loss of connection and prompted users to reconnect.

   Result: Passed: The application displayed appropriate error messages and handled internet disconnections gracefully.

2. STN-OEWE2

   Description: The product shall use a local MongoDB database hosted on the user's server or local machine for all data storage.

   Type: Functional, Dynamic, Manual

   Initial State: The web application is configured to connect to a local MongoDB instance.

   Input/Condition: User performs data storage and retrieval operations.

   Output/Result: The application successfully connects to the local MongoDB instance without external dependencies.

   How the test was performed: Testers verified the connection to the local MongoDB database and performed various data storage and retrieval operations, ensuring that all operations were successful without relying on cloud services.

   Result: Passed: The application successfully connected to the local MongoDB instance and operated as expected.

4.0.16 Requirements for Interfacing with Adjacent Systems

1. STN-OEIA1

   Description: The product shall allow users to export and import data using common formats such as CSV, Excel, and JSON.

   Type: Functional, Dynamic, Manual

   Initial State: The web application is running and a user has data available for export/import.

   Input/Condition: User exports/imports data in CSV, Excel, or JSON formats.

   Output/Result: Data should be successfully exported and imported without errors.

   How the test was performed: Testers exported and imported simulation data, model configurations, and results in various formats, verifying that the data remained intact and properly formatted.

   Result: Passed: The data was successfully exported and imported without errors.

2. STN-OEIA2

   Description: The product shall support API integration for data exchange with third-party tools.

   Type: Functional, Dynamic, Manual

   Initial State: The web application is running and configured for API access.

   Input/Condition: User performs API calls to third-party tools.

   Output/Result: Data should be exchanged seamlessly with external tools.

   How the test was performed: Testers initiated API calls to third-party tools and verified that the application successfully sent and received data, ensuring that integration worked without manual data transfers.

   Result: Passed: API integration was successful, and data exchange with third-party tools was seamless.

4.0.17 Productization Requirements

1. STN-OEP1

   Description: The product shall be distributed as a local executable that can be installed and run directly on a user's device.

   Type: Functional, Static, Manual

   Initial State: The installer for the product is available.

   Input/Condition: User downloads and installs the product.

   Output/Result: The product should run directly from the user's device without additional configurations.

   How the test was performed: Testers downloaded and installed the product, verifying that it ran successfully without requiring any additional server configurations or dependencies.

   Result: Passed: The product installed and ran successfully without additional configurations.

4.0.18 Release Requirements

1. STN-OER1

   Description: The product shall be released with full documentation, including an online help center, API documentation, and user guides.

   Type: Functional, Static, Manual

   Initial State: The product is released.

   Input/Condition: User accesses the help center and documentation.

   Output/Result: All documentation should be available and complete.

   How the test was performed: Testers navigated through the online help center, API documentation, and user guides, ensuring that all sections were accessible and provided complete information.

   Result: Passed: We haven't officially released the product. But we shared the documentation created using Microsoft Word with Dr. Mahalec's research team and they were able to operate the system based on the documentation. The documentation was complete and provided all necessary information.

4.0.19 Maintenance Requirements

1. STN-MSM1

   Description: All system maintenance must be completed at least one week prior to the release or any critical deployment.

   Type: Functional, Static, Manual

   Initial State: The system is in the pre-release phase, with a scheduled deployment date.

   Input/Condition: The deployment date is set, and no updates, patches, or changes are applied within one week of this date.

   Output/Result: No system updates, patches, or changes are made within one week of a critical release or deployment.

How the test was performed: Change logs and maintenance records were reviewed to confirm that no modifications to the system occurred within one week of the scheduled release or critical deployment date.

Result: Passed: No system updates or changes were made within the required timeframe.

2. STN-MSM2

   Description: Maintenance tasks shall be logged and tracked using an issue management system such as GitHub Issues.

   Type: Functional, Dynamic, Manual

   Initial State: Maintenance tasks are ongoing and require tracking.

   Input/Condition: Maintenance tasks are entered into the issue management system.

   Output/Result: At least 90% of maintenance tasks are logged in the issue management system, with status updates visible to developers and users.

   How the test was performed: The issue management system was reviewed, and the percentage of logged maintenance tasks was calculated to ensure at least 90% documentation with current statuses.

   Result: Passed: Over 90% of maintenance tasks were logged and properly tracked. We continuously used GitHub Issues to track all maintenance tasks.

4.0.20 Supportability Requirements

1. STN-MSS1

   Description: Comprehensive technical documentation shall be provided, covering installation, configuration, usage, troubleshooting, and best practices.

   Type: Functional, Static, Manual

   Initial State: The documentation is available to users after the system release.

Input/Condition: Users are asked to operate the system solely based on the provided documentation.

Output/Result: At least 80% of users report they can operate the system independently based on the documentation.

How the test was performed: A post-release survey of users was conducted to gather feedback on their experience using the documentation, aiming to achieve at least 80% satisfaction in users' ability to operate the system independently.

Result: Passed: We haven't officially released the product. But Dr. Mahalec's research team was able to operate the system based on the documentation. So 100% of users were able to operate the system independently based on the documentation.

4.0.21 Adaptability Requirements

1. STN-MSA1

   Description: The system shall be modular, allowing future developers to add new features or make changes to the existing codebase with minimal effort.

   Type: Functional, Dynamic, Manual

   Initial State: The system is operational, and new feature integration is being considered.

   Input/Condition: A new module or feature is developed and integrated into the existing codebase.

   Output The new module or feature integrates into the system without requiring major changes to existing code.

   How the test will be performed: Review developer inspection reports and codebase changes related to the integration, verifying that major code modifications were not necessary and modularity is maintained.

   Result Passed: All PR's are relatively simple indicating modular work

2. STN-MSA2

Description: The system shall be built using industry-standard tools and frameworks to ensure future developers can easily adapt and work on the project.

Type: Functional, Static, Manual

Initial State: The system is fully built and operational.

Input/Condition: Developers assess the tools and frameworks used in the system.

Output The system uses widely recognized technologies, facilitating future maintenance and adaptability.

How the test will be performed: Review the system's technology stack to confirm it utilizes widely adopted tools such as React, Node.js, and MongoDB, ensuring new developers can familiarize themselves with the project quickly.

Result Passed: All dependencies used in the project are widely used and installed based off of metrics on https://www.npmjs.com/

4.0.22 Access Requirements

1. ~~STN-SEC-ACC1~~

   Description: ~~The system shall enforce role-based access control (RBAC) to restrict access based on user roles.~~

   Type: ~~Functional, Dynamic, Manual~~

   Initial State: ~~The system is operational with predefined roles (Administrator, Operator, Viewer).~~

   Input/Condition: ~~Attempt access with different user roles to verify access restrictions.~~

   Output: ~~Each role can only access features and data relevant to its permissions.~~

   How the test will be performed: ~~Log in as different roles and attempt to access restricted features and data. Verify that access is limited according to each role's permissions.~~

Result: ~~Failed: While authentication was developed, role-based authentication has not been completed yet. However, we will still maintain this test/requirement since this is still a feature in active development.~~

Rev 1 explanation: This requirement has been removed from the scope of our rev 1 since our sponsor no longer needs us to have role based authentication within the scope of this project.

4.0.23 Integrity Requirements

1. STN-SEC-INT1

   Description: The system shall ensure data integrity by validating and sanitizing all input fields.

   Type: Functional, Dynamic, Automated

   Initial State: Input fields are empty.

   Input/Condition: Enter various valid and invalid data (e.g., scripts, special characters, SQL statements) into input fields.

   Output: Invalid or potentially harmful inputs are sanitized or rejected according to validation rules.

   How the test will be performed: Execute automated tests to validate and sanitize inputs. Manually test edge cases to ensure scripts and SQL injection attempts are rejected.

   Result: Passed: Automated backend unit tests and manual frontend tests indicate that all inputs are safe and secure and cannot compromise the system

2. STN-SEC-INT2

   Description: The system shall implement version control for configuration files and database entries.

   Type: Functional, Static, Automated

   Initial State: Configuration files and database entries are tracked.

   Input/Condition: Modify configuration files and database entries.

Output: Changes are tracked with timestamps, unique identifiers, and can be reverted if necessary.

How the test will be performed: Review version control logs for changes. Confirm that each change has a timestamp and unique identifier and verify the ability to revert to previous versions.

Result: Passed: All changes to the db can viewed via the MongoDB control panel and can be safely rolled back using the MongoDB interface

4.0.24 Privacy Requirements

1. STN-SEC-PRI1

   Description: The system shall implement data encryption for sensitive user information in transit and at rest.

   Type: Functional, Static, Automated

   Initial State: System with sensitive data (e.g., user credentials).

   Input/Condition: Attempt data transmission and access storage to verify encryption.

   Output/Result: Sensitive data is encrypted using AES-256 at rest and SSL/TLS during transmission.

   How the test will be performed: Inspect the encryption setup in the system for AES-256 and SSL/TLS configurations. Attempt to intercept data during transit and retrieve data from storage to verify encryption.

   Result: Passed: All passwords are properly hashed in the database to avoid potential leaks in transit and passwords cannot be seen/accessed at any point

4.0.25 Audit Requirements

1. STN-SEC-AUD1

   Description: The system shall log relevant user actions, including login attempts, data access, and modifications.

   Type: Functional, Dynamic, Automated

Initial State: The system is in use with active users.

Input/Condition: Perform user actions, such as logging in, accessing data, and making modifications.

Output: Actions are logged with timestamps, user IDs, IP addresses, and action types, stored securely for periodic review.

How the test will be performed: Check the system logs for records of each user action, including all required information. Confirm that logs are securely stored and accessible for audit purposes.

Result: Passed: All actions are appropriately logged via the terminal and can be easily restored/accessed via the src/logs directory

4.0.26 Immunity Requirements

1. STN-SEC-IMM1

   Description: The system shall employ secure coding practices and automated security testing to detect and prevent vulnerabilities.

   Type: Functional, Dynamic, Automated

   Initial State: System codebase is accessible for testing.

   Input/Condition: Run security testing tools (e.g., SAST, DAST) on the codebase.

   Output: No critical vulnerabilities are found in the code, and all identified issues are resolved before release.

   How the test will be performed: Integrate SAST and DAST tools into the development process, generating security reports. Verify that no critical issues remain unresolved.

   Result: Passed: Performed SAST scans on local development environment and no critical issues are found.

4.0.27 Cultural Requirements

1. STN-CR1

Description: The product shall consider cultural practices in an industrial setting, with English as the interface language and appropriate symbols, colors, and terminology.

Type: Functional, Static, Manual

Initial State: The system interface is finalized.

Input/Condition: Review interface symbols, colors, and terminology.

Output: All elements are appropriate for an industrial environment and understandable by English-speaking users.

How the test will be performed: Conduct a cultural review of the interface to ensure compliance, consulting with industry experts if necessary.

Result: Passed: Cultural review with chemical engineering PHD students indicated that the system complied with their cultural requirements.

4.0.28 Legal Requirements

1. STN-LR1

Description: The product shall comply with IP protection laws, transferring IP to Dr. Mahalec under a proprietary license and ensuring data security compliance.

Type: Non-Functional, Static, Manual

Initial State: Product is ready for final release.

Input/Condition: Conduct a legal review for IP transfer and data security compliance.

Output: IP is transferred under a proprietary license, and no unauthorized data access is possible.

How the test will be performed: Consult with a legal team to confirm IP compliance and review security documentation to verify that data security standards are met.

Result: Passed: Professor Mahelec provided us with a legally binding contract and we included a license indicating the propietary nature of our software in this repository.

2. STN-LR2

   Description: The system shall not use any copyrighted or patented technology without proper licensing or permission.

   Type: Non-Functional, Static, Manual

   Initial State: All third-party components are documented.

   Input/Condition: Review third-party licenses and permissions for all integrated technologies.

   Output: Documentation confirms all third-party components are properly licensed for commercial use.

   How the test will be performed: Conduct a thorough review of third-party licenses, verifying all are documented and properly licensed for commercial use.

   Result: Passed: All dependency and third party software used in the system are allowed for commercial use.

4.0.29 Standards Compliance Requirements

1. No Specific Standards Compliance Requirement

   Description: Since no specific industry or software standards apply, no standard compliance testing is required.

   Type: Not Applicable

   Initial State: Not Applicable

   Input/Condition: Not Applicable

   Output: Not Applicable

   How the test will be performed: Not Applicable

   Result: Not Applicable

5 Comparison to Existing Implementation

Our project is completely new so this section is not applicable.

6 Unit Testing

The unit tests for this application are performed using Jest, a testing framework written in TypeScript. The APIs are tested without actually making calls to the database, which is performed by building mock versions of the database, in the beforeAll function.

6.1 Authentication Routes

6.1.1 Signup - Create a New User

Initial State: The database is initialized, and the system is ready to accept user signup requests.
Input: A POST request to /api/auth/signup with the following payload:

```
{ "email": "newuser@example.com", "password": "newpassword123", "firstName": "New
```

Expected Output: The response should have a status code of 201 and a message:

```
"User created successfully"
```

Actual Output: User created successfully.
Result: Pass - The user was successfully created with the correct response.

6.1.2 Login - Return Access and Refresh Tokens

Initial State: A user exists in the database with valid credentials.
Input: A POST request to /api/auth/login with the following payload:

```
{ "email": "test@example.com", "password": "password123" }
```

Expected Output: The response should have a status code of 200 and return both an access token and a refresh token.
Actual Output: Access and refresh tokens returned successfully.
Result: Pass - Tokens returned successfully, as expected.

6.1.3 Login - Fail with Incorrect Password

Initial State: A user exists in the database, but an incorrect password is provided.
Input: A POST request to /api/auth/login with the following payload:

```
{ "email": "test@example.com", "password": "wrongpassword" }
```

Expected Output: The response should have a status code of 401 and return the message:

```
"Invalid email or password"
```

Actual Output: Authentication failed with the expected message.
Result: Pass - The system correctly denied access with the expected error message.

6.1.4 Logout - Remove Refresh Token

Initial State: A user is logged in and has an active refresh token stored.
Input: A POST request to /api/auth/logout with the authorization header containing a valid access token.
Expected Output: The response should have a status code of 200 and return the message:

```
"Logged out successfully"
```

Actual Output: Logout successful, refresh token removed.
Result: Pass - The refresh token was successfully removed and the logout completed.

6.2 Data Routes

6.2.1 Data Retrieval - Fetch All Data

Initial State: The system is initialized with test data.
Input: Valid authentication token and a GET request to `/data`.
Expected Output: A list of all data entries in JSON format.
Actual Output: A list of all data entries in JSON format.
Result: Pass - The system returned all data entries as expected.

6.2.2 Test Unauthorized Data Access

Initial State: The system is running, but no authentication token is provided.
Input: GET request to `/data` without authentication.
Expected Output: The system returns an unauthorized access error.
Actual Output: The system returns an unauthorized access error.
Result: Pass - Unauthorized access was correctly denied.

6.2.3 Data Manipulation - Test Add New Data

Initial State: The system is ready, and authentication is provided.
Input: POST request to `/data` with valid data payload.
Expected Output: Data is successfully added, and a confirmation response is returned.
Actual Output: Data is successfully added, and a confirmation response is returned.
Result: Failed - Adding an empty data object results in an error
Rev 1 Result: Passed: This bug was fixed by adding validation for empty objects into the function that handles new data additions

6.2.4 Data Manipulation - Test Update Existing Data

Initial State: The system contains test data, and authentication is provided.
Input: PUT request to `/data/:id` with updated values.
Expected Output: The specified data entry is updated successfully.
Actual Output: The specified data entry is updated successfully.
Result: Pass - The data entry was correctly updated.

6.2.5 Data Manipulation - Test Delete Data Entry

Initial State: The system contains test data, and authentication is provided.
Input: DELETE request to `/data/:id`.
Expected Output: The specified data entry is removed.
Actual Output: The specified data entry is removed.
Result: Pass - The data entry was successfully deleted.

7 Changes Due to Testing

7.1 Changes due to STN-LAFSR3 failing

STN-LAFSR3 faied due to edges not having the desired level of interactivity. When the user would select an edge there would be no visual feedback for the user indicating that the edge was selecting. We fixed this after the test by adding a highlight animation whenever an edge was selected to make it clear that an edge was selected to the user allowing us to pass this test on a second attempt.

7.2 Changes due to STN-UHE1 failing

STN-UHE1 was a test that evaluated the clarity of different components in the UI by checking to see if the components are well labeled via a manual test. However the tet failed due to certain UI components (specifically input fields in the node modals) missing labels which made their use unclear to the end user. We fixed this by going back into the codebase and adding labels to all elements that were found to be missing labels in the test. We then ran the test a second time and passed the one to one label to component evaluation.

7.3 ~~Changes due to STN-SEC-ACC1 failing~~

~~This test failed due to not implementing any sort of role based authentication yet. We still intend to have this developed by the time we submit the final project so this failure has not resulted in any direct changes and we will not be removing this test case or requirement.~~ This change is no longer needed since this requirement is no longer within the scope of this project.

7.4 Changes due to 6.2.3 failing

This test was a failure in a unit test where the system fails to accept an empty data input. We fixed this by adding proper input sanitization so that our system is able to accept empty or malformed data inputs and provide an appropriate error response rather than a system crash.

7.5 Rev 1 update:

After the bugs/issues found in the vnv report, the above changes were made for rev 1. In rev 1 3/4 issues were fixed while the fourth problem (role based authentication), was removed from the scope of the project and so there are no more changes remaining.

8 Automated Testing

8.1 Automated Testing for Backend

Automated testing for our Node and Express backend is done using Jest along with `supertest`. All backend related test files `check.test.ts`, `dataRoutes.test.ts`, and `userRoutes.test.ts` are located directly under the `tests/` folder. These tests perform HTTP requests against our API endpoints and use `expect` statements to verify that the server responses match the expected status codes and response.

8.2 Automated Testing for Frontend

For the frontend, we used Selenium IDE, which relies on a click-based (record-and-playback) testing approach. All of these Selenium test files such as `capstone.side`, `CreateDiagram.spec.ts`, `Login.spec.ts`, `Logout.spec.ts`, `SaveDiagram.spec.ts`, and `Signup.spec.ts` can be found in the `tests/frontend/` folder. This setup makes sure that common user flows like signup, login, logout, creating and saving diagrams are automatically tested via form interactions and click events. Features that require precise or more complex interactions like drag-and-drop within the React Flow canvas are manually tested due to the limitations of simple click-based tools.

8.3 Frontend Selenium Click-Based Testing

8.3.1 Signup

Initial State: The application is running on `localhost:5173`, and the user has not registered yet.
Input:

- Open `/login` page in browser.

- Click the `Sign Up` link.
- Enter "Test1" in the First Name field.
- Enter "Test2" in the Last Name field.
- Enter "Test99@demo.com" in the Email field.
- Enter "1234" in the Password field.
- Click the "Sign Up" button.

Expected Output: The user is redirected to a page that displays the heading "Login."
Actual Output: The heading "Login" is displayed immediately after clicking Sign Up.
Result: Pass - The user is successfully registered and redirected.

8.3.2 Login

Initial State: A user with credentials (`email: Test99@demo.com, password: 1234`) already exists.
Input:

- Open `/login` page.
- Type "Test99@demo.com" in the Email field.
- Type "1234" in the Password field.
- Click the "Login" button.

Expected Output: The browser navigates to the Dashboard, showing "Create New Network."
Actual Output: The Dashboard is displayed with the heading "Create New Network."
Result: Pass - The user is successfully logged in and sees the expected page.

8.3.3 Logout

Initial State: The user is currently logged in and on the Dashboard page.
Input:

- Navigate to `/dashboard`.
- Click the "Logout" button (CSS class `.btn-danger`).

Expected Output: The application redirects to the login page with the heading "Login."
Actual Output: The heading "Login" is shown after clicking Logout.
Result: Pass - The user is logged out and returned to the login screen.

8.3.4 Create Diagram

Initial State: The user is logged in and is at the Dashboard (`/dashboard`).
Input:

- Click the `canvasName` field and type "Test1234."
- Select "Hydrogen Plant" from the Domain dropdown.
- Select `sim` from the Calculation Type dropdown.
- Enter "Hello Testing" in the Description text area.
- Click the "Create Diagram" button.

Expected Output: The page transitions to the Canvas view, which displays a "Model Pallete" sidebar.
Actual Output: The application navigates to the Canvas view, and the sidebar with "Model Pallete" is confirmed.
Result: Pass - Diagram creation flow works as intended.

8.3.5 Save Diagram

Initial State: The user is on the Dashboard, ready to create and then save a diagram.
Input:

- Type "Test1" in the `canvasName` field.

- Choose “Hydrogen Plant” as the Domain.
- Select `sim` for the Calculation Type.
- Enter “1234” in the Description.
- Click the “Create Diagram” button to open the Canvas.
- Once on the Canvas page, click “Save Network.”

Expected Output: A button labeled “Verify Diagram” is present, indicating the diagram has been saved (and can be verified later).
Actual Output: The “Verify Diagram” button appears, confirming that the save action was performed.
Result: Pass - The diagram is saved successfully and the correct button is shown.

9 Trace to Requirements

Table 1: Functional Requirements Traceability Matrix

Test Cases	**FR1**	**FR2**	**FR3**	**FR4**	**FR5**	**FR6**	**FR7**	**FR8**	**FR9**	**FR10**
STN-FR1	✓									
STN-FR2	✓									
STN-FR3	✓									
TSTN-FR4		✓								
STN-FR5		✓								
STN-FR6			✓							
STN-FR7				✓						
STN-FR8					✓					
STN-FR9						✓				
STN-FR10							✓			
STN-FR11								✓		
STN-FR12									✓	
STN-FR13										✓

Table 2: Non-Functional Requirements Traceability Matrix (Part 1)

Test Cases	**LAFAR1**	**LAFAR2**	**LAFAR3**	**LAFSR1**	**LAFSR2**	**LAFSR3**	**UHE1**	**UHE2**	**UHPI1**	**UHL1**	**UHL2**	**UHU1**
STN-LAFAR-1	✓											
STN-LAFAR-2		✓										
STN-LAFAR-3			✓									
STN-LAFSR-1				✓								
STN-LAFSR-2					✓							
STN-LAFSR-3						✓						
STN-UHE-1							✓					
STN-UHE-2								✓				
STN-UHPI-1									✓			
STN-UHL-1										✓		
STN-UHL-2											✓	
STN-UHU-1												✓

Table 3: Non-Functional Requirements Traceability Matrix (Part 2)

Test Cases	**PS1**	**PS2**	**PS3**	**PS4**	**PPA1**	**PPA2**	**PRF1**	**PRF2**	**PC1**	**PSE1**	**PL1**	**UHU-1**	**UHA-1**
STN-PS1	✓												
STN-PS2		✓											
STN-PS3			✓										
STN-PS4				✓									
STN-PPA1					✓								
STN-PPA2						✓							
STN-PRF1							✓						
STN-PRF2								✓					
STN-PC1									✓				
STN-PSE1										✓			
STN-PL1											✓		
STN-UHU-1												✓	
STN-UHA-1													✓

Table 4: Non-Functional Requirements Traceability Matrix (Part 3)

Test Cases	**OEEP1**	**OEEP2**	**OEEP3**	**OEWE1**	**OEWE2**	**OEIA1**	**OEIA2**	**OEP1**	**OER1**	**MSM1**	**MSM2**	**MSS1**
STN-OEEP1	✓											
STN-OEEP2		✓										
STN-OEEP3			✓									
STN-OEWE1				✓								
STN-OEWE2					✓							
STN-OEIA1						✓						
STN-OEIA2							✓					
STN-OEP1								✓				
STN-OER1									✓			
STN-MSM1										✓		
STN-MSM2											✓	
STN-MSS1												✓

Table 5: Non-Functional Requirements Traceability Matrix (Part 4)

Test Cases	**MSA1**	**MSA2**	**SEC-ACC1**	**SEC-INT1**	**SEC-INT2**	**SEC-PRI1**	**SEC-AUD1**	**SEC-IMM1**	**CR1**	**LR1**	**LR2**
STN-MSA1	✓										
STN-MSA2		✓									
~~STN-SEC-ACC1~~			~~✓~~								
STN-SEC-INT1				✓							
STN-SEC-INT2					✓						
STN-SEC-PRI1						✓					
STN-SEC-AUD1							✓				
STN-SEC-IMM1								✓			
STN-CR1									✓		
STN-LR1										✓	
STN-LR2											✓

10 Trace to Modules

Table 6: Module Traceability Matrix (Part 1)

Test Cases	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**
STN-FR1					✓	✓			
STN-FR2					✓	✓			
STN-FR3					✓	✓			
STN-FR4					✓	✓			
STN-FR5					✓	✓			
STN-FR6							✓		✓
STN-FR7					✓		✓		✓
STN-FR8		✓	✓						
STN-FR9		✓		✓				✓	
STN-FR10					✓		✓		
STN-FR11					✓				✓
STN-FR12			✓		✓		✓		✓
STN-FR13			✓		✓				✓

Table 7: Module Traceability Matrix (Part 2)

Test Cases	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**
STN-LAFAR-1					✓				
STN-LAFAR-2		✓			✓			✓	
STN-LAFAR-3					✓				
STN-LAFSR-1		✓			✓			✓	
STN-LAFSR-2		✓			✓			✓	
STN-LAFSR-3		✓			✓			✓	
STN-UHE-1		✓			✓			✓	
STN-UHE-2					✓				
STN-UHPI-1		✓			✓			✓	
STN-UHL-1		✓	✓	✓	✓	✓	✓	✓	
STN-UHL-2		✓			✓			✓	
STN-UHU-1					✓			✓	
STN-UHA-1	✓								

Table 8: Module Traceability Matrix (Part 3)

Test Cases	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**
STN-PS1		✓	✓	✓	✓			✓	
STN-PS2			✓				✓		
STN-PS3				✓			✓		
STN-PS4				✓		✓		✓	
STN-PPA1					✓				
STN-PPA2					✓				
STN-PRF1					✓		✓		✓
STN-PRF2		✓	✓	✓	✓			✓	
STN-PC1			✓	✓	✓		✓		✓
STN-PSE1		✓					✓		✓
STN-PL1		✓	✓	✓	✓	✓	✓	✓	

Table 9: Module Traceability Matrix (Part 4)

Test Cases	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**
STN-OEEP1	✓								
STN-OEEP2	✓								
STN-OEEP3	✓	✓			✓			✓	
STN-OEWE1	✓	✓	✓	✓		✓	✓		✓
STN-OEWE2							✓		
STN-OEIA1							✓	✓	
STN-OEIA2							✓		
STN-OEP1		✓	✓	✓	✓	✓	✓	✓	
STN-OER1		✓	✓	✓	✓	✓	✓	✓	
STN-MSM1		✓	✓	✓	✓	✓	✓	✓	
STN-MSM2		✓	✓	✓	✓	✓	✓	✓	
STN-MSS1		✓	✓	✓	✓	✓	✓	✓	

Table 10: Module Traceability Matrix (Part 5)

Test Cases	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**
STN-MSA1		✓	✓	✓	✓	✓	✓	✓	
STN-MSA2		✓	✓	✓	✓	✓	✓	✓	
~~STN-SEC-ACC1~~		~~✓~~					~~✓~~		
STN-SEC-INT1		✓			✓			✓	
STN-SEC-INT2						✓	✓		
STN-SEC-PRI1		✓					✓		
STN-SEC-AUD1		✓	✓	✓		✓	✓		
STN-SEC-IMM1		✓	✓	✓	✓	✓	✓	✓	
STN-CR1		✓			✓			✓	
STN-LR1		✓	✓	✓	✓	✓	✓	✓	
STN-LR2		✓	✓	✓	✓	✓	✓	✓	

11 Code Coverage Metrics

The code coverage was run on all of the backend codebase using Jest's built-in flag "–coverage". The code coverage metrics are shown in the screenshot below:

```
Kishor Pandya@DESKTOP-2SM9U4V MINGW64 /d/McMaster/Engineering/SoftwareEng/4G06/capstone/src (main)
$ npm test

> Plant-GUI@1.0.0 test
> jest --coverage

 PASS  tests/dataRoutes.test.ts (19.463 s)
  ● Console

    console.info
      Reading system DB

      at info (src/backend/routes/dataRoutes.ts:204:13)

 PASS  tests/userRoutes.test.ts (19.518 s)
 PASS  tests/check.test.ts (19.66 s)
A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with --detectOpenHandles to find leaks. Active timers can also cause t
his, ensure that .unref() was called on them.
-------------------|---------|----------|---------|---------|-------------------------------------------------------------------------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------------------------------------------------------------------
All files          |   51.71 |    18.09 |   70.21 |   51.62 |
 backend           |   71.11 |    55.55 |      40 |   70.45 |
  app.ts           |   72.72 |    42.85 |   33.33 |   71.87 | 26-29,47-48,54-64
  server.ts        |       0 |      100 |       0 |       0 | 1-5
  swagger.ts       |     100 |      100 |     100 |     100 |
 backend/config    |     100 |      100 |     100 |     100 |
  prismaClients.ts |     100 |      100 |     100 |     100 |
 backend/middleware |   73.68 |       60 |     100 |   72.22 |
  auth.ts          |   73.68 |       60 |     100 |   72.22 | 14-15,26-27,35
 backend/routes    |   52.01 |     38.4 |    72.5 |   51.68 |
  dataRoutes.ts    |    49.5 |     42.3 |      75 |   49.25 | ...255,263-264,268,311,357,388-411,457-519,551-595,604-639,689-690,721,761-764
  index.ts         |     100 |      100 |     100 |     100 |
  userRoutes.ts    |   52.87 |    19.04 |   57.14 |   52.32 | 16-17,51-52,88-89,94-116,121-163,172-175,185-186
 backend/types     |       0 |        0 |       0 |       0 |
  express.d.ts     |       0 |        0 |       0 |       0 |
 backend/utils     |   14.28 |     0.56 |     100 |   15.38 |
  logger.ts        |     100 |      100 |     100 |     100 |
  translation.ts   |    7.69 |     0.56 |     100 |    8.33 | 127-376
-------------------|---------|----------|---------|---------|-------------------------------------------------------------------------------

Test Suites: 3 passed, 3 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        29.096 s
Ran all test suites.
```

If more you need more information on the code coverage, you need to run “npm test” and run the “index.html” file in the “coverage” folder which is autogenerated.

The reason the coverage is low is because we do not test all the endpoints since our frontend does not use all the endpoints opened by our backend. We have only tested the ones actively used by our frontend. Additionally, the frontend is tested using automated click-based testing which is not covered by the unit tests.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
   Kishor: Since our SRS document was well formatted and complete, it was easy to map the requirements to the tests.
   Dhrumil: The structure and format were clear from the start, which made it easy to keep everything consistent. Plus, the test cases were well-documented, so running and reporting them was straightforward.
   Madhu: Carrying out the manual tests with appropriate testers went well since our sponsor is heavily involved in the project.
   Rasik: Given the VNV plan a lot of the work creating the tables and test cases was done beforehand and quite reusable. This was very helpful to completing the work on a timely manner and it allowed us to focus on performing the tests

2. What pain points did you experience during this deliverable, and how did you resolve them?
   Kishor: There were way too many non-functional requirements. It just got very boring after a bit.
   Dhrumil: Some tests needed adjustments to be practical for manual execution. To fix this, we introduced alternative monitoring methods,

like tracking system resources and measuring response times.
Madhu: Some of our tests felt repetitive since some of our requirements were closely related and therefore making tests for each requirement can become a little repetitive.
Rasik: Working with outside testers was pain inducing, as they're not aware of the functionalities of the application.

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?
   Our client wanted automated tests and so the unit testing/automated testing stemmed from our client. On the other hand we came up with the manual nfr and functional tests independantly since we found it important to have a set of manual tests to evaluate our ability to fulfill the system requirements. Similarly we came up with the changes based off of the test independantly since we were able to evaluate the causes of the failures in the test and use them to come up with appropriate changes/solutions.

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)
   In our VnV plan we initially planned on having a larger suit of unit tests and automated tests as we expected to use our unit tests/automated tests to provide full coverage of our frontend codebase. However in practice we realized that our frontend was too complicated and the codebase was too large to effectively test it with full code coverage using just unit tests. This is why we eventually swapped to a more manual testing based approach in the VnV report and did not have as many unit tests. Instead we mainly just used unit tests for our backend systems and relied on manual testing for the frontend. In the future we can avoid this issue by setting appropriate expectations on the level

of complexity of the codebase and be realistic about how many code based unit tests we can setup for a complex codebase.